

Howzatt! Cricket Scorekeeper

Shreshtha Gupta
Roll No. 24B1033

Contents

1	Introduction	2
2	How to Use Howzatt! Cricket Score Keeper	2
2.1	Step 1: Match Setup	2
2.2	Step 2: Live Match Tracking	2
2.3	Step 3: Viewing the Scorecard	3
2.4	Step 4: Match Ending	3
3	Project Structure	3
4	DETAILS OF BASIC FEATURES IMPLEMENTED	4
4.1	Match Setup Page	4
4.2	Live Match Interface Working	4
4.2.1	Global Variables Initialization	5
4.2.2	Functions Logic:	5
4.3	Scorecard	8
4.4	Match Summary	9
4.5	CSS	10
5	BRIEF DETAILS OF CUSTOMIZATION	11
5.1	Wide	11
5.2	No-ball	11
5.3	Byes and leg-byes	12
5.4	Live Commentary	12
5.5	Chase-Box	14
5.6	Background Music	14
5.7	More Extensions for Basic Implementations	14
6	References	15

Abstract

This project involves building a simple live cricket-scoring web application using HTML, CSS, and JavaScript. The web app will allow a scorer to input match events (runs, extras, wickets, etc.) using buttons, and the website will automatically update player and match statistics in real-time.

1 Introduction

"Howzatt!" lets users take charge of cricket matches between two teams — from setting up the match details to live scoring every ball, and finally viewing a detailed scorecard and result summary. The entire application is built using HTML, CSS, and JavaScript, with all data securely handled through client-side `localStorage` and is saved in the local storage so it won't be reset until and unless you want to.

2 How to Use Howzatt! Cricket Score Keeper

If you want to play a match not for 2 overs, you can change the variable at the very beginning of code (Line no. 40) as the whole logic code is **not hardcoded** for 2 overs.

2.1 Step 1: Match Setup

- Open `setup.html`.
- Enter the following details:
 - Team 1 Name
 - Team 2 Name
 - Toss Winner(who won the toss - Team 1 or Team 2)
 - Toss Decision(what they chose - Batting or Bowling)
- Click on **Start Match** to begin the match.

2.2 Step 2: Live Match Tracking

- You will be redirected to the next Live Match page (`live.html`).
- Use the buttons to record ball-by-ball events:
 - Click on 0, 1, 2, 3, 4, or 6 depending on runs scored.
 - Click on **Wicket** if a batter gets out.
 - Click on **Wide**, **No Ball**, **Bye** or **legbyes** for extras.
 - To change the bowler in between an over, press **New Bowler**.

Important: Some prompts are intentionally delayed to avoid overlapping with others by `setTimeout`.

2.3 Step 3: Viewing the Scorecard

- Click on the **Scorecard** button anytime to view the current match's batting and bowling records.
- Now on the scorecard page, stats of current innings will be visible.
- Click on reset button if you want to reset the match and start a new and click on back to match to continue with live scoring.

2.4 Step 4: Match Ending

- After the first innings is completed, the app automatically sets up the second innings.
- After the second innings:
 - A match summary (win/lose/tie) will be generated and you will be redirected to the **summary** page.
 - you can click on reset match which will redirect you to **setup.html** where one can start with a fresh match.

3 Project Structure

- **setup.html / setup.css**: Take inputs of match details and store it to **localStorage**
- **live.html / live.css**: Live scoring of a match with real time sense
- **scorecard.html / scorecard.css**: Displays the batters and bowlers stats at that point of time.
- **summary.html / summary.css**: Displays summary of the match (who won or if tied) and allows to start a fresh match by reset button.
- **score.js**:
 - Update score dynamically when buttons are clicked.
 - Track overs, balls, wickets, and runs.
 - Automatically rotate strike for odd runs (1, 3, 5).
 - Prompt for strike batter's name, non-strike batter's name and first bowler's name at the beginning of the match.
 - Prompt for a new batter's name when a wicket falls.
 - Prompt for a new bowler's name at the end of an over.
 - Calculate and display Current Run Rate (CRR) and Required Run Rate (RRR) (this is applicable only in the second innings).
 - Live commentary generation
 - Emphasized Chase box during 2nd innings

4 DETAILS OF BASIC FEATURES IMPLEMENTED

4.1 Match Setup Page

`setup.html` allows users to enter the names of both teams, select the toss winner, and choose the toss decision(bat or bowl). To implement the js code of setup in `score.js`, i have added an if statement that if we are on setup page, certain set of code runs that takes inputs of team names, toss winner and decision. an error message is also handled if any of the team name is empty, it will throw an alert.

Styling of `setup.html` is handled by `setup.css` where i have added background image and a container box with button hover effects.

When the user clicks the "Start Match" button, the event listener clears any old match data from `localStorage` because of the added `localStorage.clear()` fn and saves the new match setup information as a JSON object under the key `matchData..`

After saving the match setup into `localStorage`, the page automatically redirects the user to `live.html`, where live match scoring begins but logic is handled by `score.js` only with else code block.

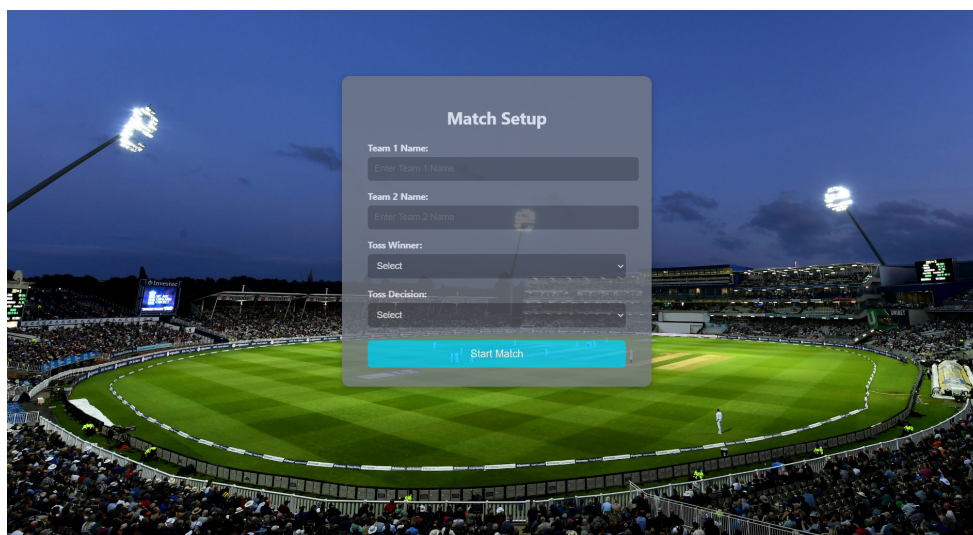


Figure 1: Setup Page: Let's start a match

4.2 Live Match Interface Working

`live.html` page basically shows stats of striker and non-striker batsman playing, and current bowler stats. It consists of certain dynamic buttons for runs scores, wicket, new bowler, extras like no ball, wide, byes, leg-byes, a button to make a way to `scorecard.html`, a container for live commentary with responsive layout. Flex containers and layout is styled by `live.css`

Display of live scores during 1st and 2nd innings is as:

`live.html` dynamically works as:

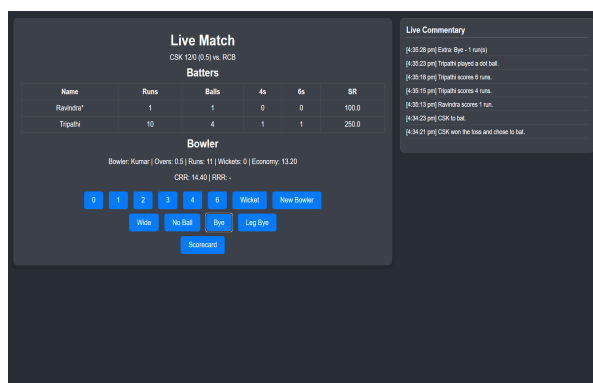


Figure 2: Innings 1

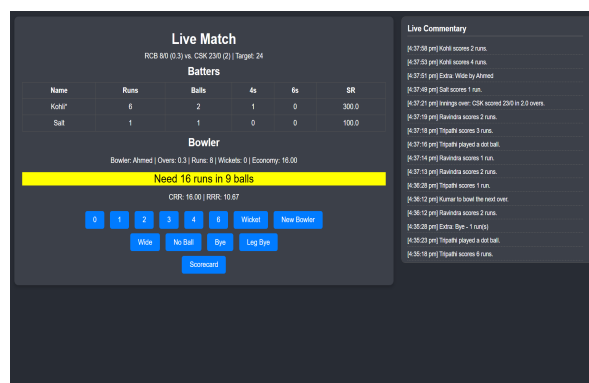


Figure 3: Innings 2

4.2.1 Global Variables Initialization

- Initializes match state variables such as score, wickets, balls, overs, striker, non-striker, currentbowler, batters, bowlers, extras, currentinnings, inningsData, target, commentary, and others.
- `gameState` object bundles all these variables for easy saving/loading.

4.2.2 Functions Logic:

- function savegame()**
 - Saves the current game state by serializing the `gameState` object into JSON object and storing it in `localStorage` under `gameState`.
 - Ensures persistent game progress across page reloads.
- function loadgame()**
 - Loads the saved game state from `localStorage` when we come back from scorecard page to live page.
 - If found, restores all game variables from the saved state.
 - If not found, starts a new game (will be possibility if directly scorecard page is opened.)
 - Calls `displayscore()` to update the UI.
- function startgame()**
 - Used the store data in `gameState.matchData` to get the values from setup page, then gave alerts showing

X won the toss and chose to Y

Z to bat.
 - Prompts users to enter opener batsmen's names and first bowlers name.(Make sure to use correct team member's names)
 - Initializes batter and bowler objects with zero stats.

- Calls `displayscore()` and `savegame()` to show the initial score and save the state.
- `function crrcalc()` and `function rrrcalc()`

- Formula used to calculate current run rate(crr) and required run rate(rrr) is:

$$CRR = \frac{\text{Total team runs}}{\text{Total overs completed}}$$

$$RRR = \frac{\text{Total team runs required to win}}{\text{Total overs remaining}}$$

- RRR is only shown in 2nd innings, During 1st innings, it only shows RRR: -
- both are rounded off to 2 decimal places by `.toFixed(2)`.
- Formula used to code the logic is as follows:

```
function crrcalc(){
return overs > 0 ? (score/ ((Math.floor(overs) +
((overs*10) % 10)/6))))).toFixed(2) : "0.00";}

function rrrcalc(){
const remballs = (overslmt * 6) - totalballs;
const removrs = remballs / 6;
return removrs > 0 ? ((target - score) / removrs).toFixed(2) : "0.00";}
```

- `function updateScore(runs)`

- Function is called directly from `live.html` within button tags with arguments as its value (1,2,3,4,6)
- Updates the score and stats when runs are scored off a legal ball.
- Increments total runs, balls, and total balls bowled.
- Updates overs in decimal formula by using:

```
overs = (Math.floor(totalballs / 6)) + (totalballs % 6) / 10;
```

- Updates striker's runs, balls faced, and boundary counts(4s and 6s).
- Updates current bowler's balls bowled and runs conceded.
- Calls `displayscore()` to update UI.
- Swaps strike if runs scored are odd.
- Changes bowler after every 6 legal balls (an over).
- Ends innings if balls reach the over limit.
- During second innings, updates chase info display and checks if target is reached to end innings. To emphasize it, i have used inline styling.
- Saves the updated game state.

- `function swapstrike()`

- Swaps the striker and non-striker batter.
- Calls `upbattable` to update batter stats.(striker batsman always comes on top of the live-table with asterisk)
- `function displayscore()`
 - Updates the score display on the UI.
 - Shows current innings team score and wickets with overs bowled.
 - display is as: During the first innings:
CSK 50/3 (1.4) vs. RCB
During the second innings,
RCB 20/1 (0.5) vs. CSK 61/3 (2.0) — Target: 62
 - Displays current run rate and required run rate (in 2nd innings).
 - Calls `upbattable()` and `upbowlers()` to update batter and bowler stats tables.
- `function upbattable()`
 - Updates the batter stats table for striker and non-striker.
 - Calculates strike rates for each batter with the formula:
$$\text{Strike Rate} = \left(\frac{\text{no of runs scored} * 100}{\text{no of balls faced by the batsman}} \right)$$
 - Displays batters name, runs, balls, fours, sixes, and strike rate.
 - highlights the striker with asterisk.
 - logic code to calculate strike rate is as:

```
const sr = (b) => b.balls ? ((b.runs / b.balls) * 100).toFixed(1) : "0.0";
```
- `function upbowlers()`
 - Updates the bowler stats(Overs, Maidens, Runs Conceded, Wickets, Economy Rate).
 - It updates economy rate with following formula:
$$\text{Economy rate} = \left(\frac{\text{no of runs conceded}}{\text{no of overs of bowler}} \right)$$
 - Logic code for calculating economy rate is

```
const eco = b.balls ? (b.runs / (b.balls / 6)).toFixed(2) : "0.00";
```
- `function wicket()`
 - Ends the current innings and transitions the match state.
 - When a wicket falls, wickets is incremented by 1.

- The innings ends function is called if wickets reaches 10 (all out) or overs limit is reached.
- `function changebowler()`
 - Called after an over is completed and also if users wants to. (manual option given - button as "New Bowler")
 - Prompts user to enter the new bowler's name and initializes its stats.
 - Ensures Bowling rotation rules and updates the UI accordingly.
 - Maidens get calculated from `function lastovrruns(bowlerName)` (it can be computed by summing runs from the last 6 balls or from the current bowler's over stats.)
- `function endinni(won = false)`
 - Saves the current innings data (score, wickets, overs, extras, batters, bowlers) into `inningsData`.
 - If first innings, sets the target for the second innings as `score + 1`.
 - If second innings, declares match complete and triggers result display.
 - Prepares the game for the next innings or finishes the match. If finishes, transfers to summary page by showing alert:

Match Over!

4.3 Scorecard

`scorecard.html` displays the detailed match summary upto that point. it has two table for displaying batters data and bowlers data. It also shows the no of extra runs scored by the team. It has a button to go back to `live.html` and a button to reset match which takes you to `setup.html`. Styling of table, buttons and body is done by `scorecard.css`.

1. The page calls the function `makesc()` which calls `geninnhtml()`
2. `function geninnhtml(title, batters, bowlers, extras, score, wickets, overs)` generates HTML for one innings:
 - Constructs a section with:
 - Innings title + total score/wickets/overs
 - A batting table showing runs, balls, 4s, 6s,strike rate
 - A bowling table showing overs, maidens, runs,wickets, economy rate
 - Extras summary
 - Returns complete HTML as a string.
3. `function makesc()` Generates the full scorecard (batting + bowling tables)
 - Fetches saved innings from `localStorage.inningsData` and `localStorage.gameState`.
 - iterates through each completed innings and passes data to `geninnhtml()` to generate HTML.

Match Scorecard					
Innings 1 (in progress): CSK - 15/0 (1.1 overs)					
Batting					
Batter	Runs	Balls	4s	6s	SR
Ravindra	3	2	0	0	150.0
Tripathi	11	5	1	1	220.0
Bowling					
Bowler	Overs	Maidens	Runs	Wickets	Economy
Kumar	1.0	0	13	0	13.00
Hazlewood	0.1	0	1	0	6.00
Extras: Wides - 0, No Balls - 0, Byes - 1, LegByes - 0					
Back to Match Reset match					

Figure 4: ScoreCard

- If a live innings exists, it creates a separate section marked as “in progress”.
- Appends all HTML tables to `#sccontainer`.
- Displays “No match data found” if no innings data exists.

4.4 Match Summary

`summary.html` It displays the final result of the match and has a reset button. Its basic styling of body and button is done by `summary.css`

1. The final result of the result is displayed in the format:
 If Team A wins batting first:
 ”Team A wins by X runs!”
 If Team B wins chasing:
 ”Team B wins by X wickets (Y balls left)!”
2. `funciton makesumm()` Generates match summary and result
 - Fetches both innings from `localStorage.inningsData` and clears match data.
 - Compares scores to:
 - Declare winning team, win margin (by wickets or runs), and balls left.
 - Or state match tied if scores equal.
 - Constructs a readable summary showing:
 - Result statement

- Brief stats (score, overs, extras) for both innings
 - Injects this into #summaryContainer.
 - if clicks on reset button, calls reset function.
3. `function resetfn()` (event listener)
 - Connected to a reset button on scorecard/summary pages.
 - Intended to trigger a function (`resetfn`) to reset match state
 4. Display is like If Team A wins batting first: "Team A wins by X runs!" or if Team B wins chasing: "Team B wins by X wickets (Y balls left)! All data taken from `inningsData`

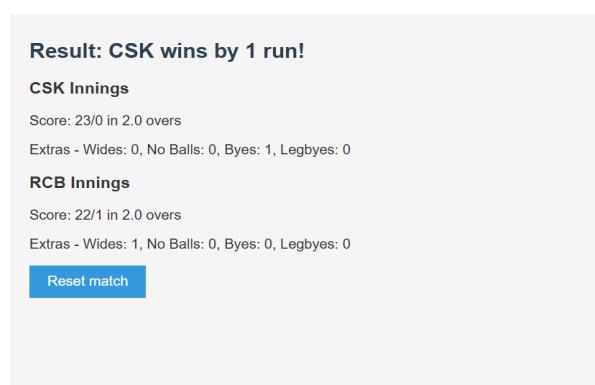


Figure 5: Summary: TEAM 1 wins batting first

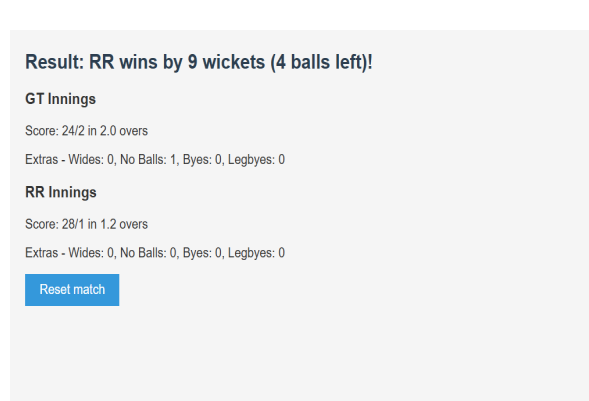


Figure 6: Summary: Team B wins chasing

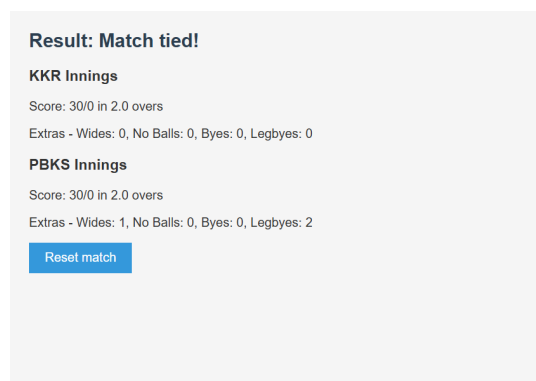


Figure 7: Summary: Match Tied

4.5 CSS

1. `setup.css` styles the `setup.html` for form, select and input types and specific paragraphs, also a background image, and i have made the opacity for form box container as 0.8
2. `live.css` styles `live.html` for table and commentary box. i have made flex containers to show live table and commentary list box side-by-side.
3. `scorecard.css` styles `scorecard.html` for tables and showing extras.

4. summary.css styles summary.html for paragraph and reset button.
5. I have given hover effects for buttons with button:hover
6. To align the boxes in the page, padding, margin and gap is used.

5 BRIEF DETAILS OF CUSTOMIZATION

5.1 Wide

When the wide button is clicked, it calls extraBall(type) function with "wide" argument which does:

1. Adds 1 run to the batting team's total score for the wide.
2. Attributes that 1 run to the current bowler's conceded runs.
3. Increments the count of wide balls in the extras object.
4. Refreshes scoreboard display and persists updated state to local storage.

5.2 No-ball

When the wide button is clicked, it calls extraBall(type) function with "no" argument which does:

1. Adds 1 run to the batting team for the no-ball itself.
2. Adds the no-ball run to the current bowler's conceded runs.
3. Increments the count of no balls in the extras object.
4. When a no ball is detected (type === 'no'), a free hit is triggered, and the following logic captures and processes runs scored from it:
 - The user manually inputs what happened on the free hit in prompt: 1,2,3,4,5,6 for runs and 'W' or 'w' for wicket.
 - If wicket is entered, it will work as a normal wicket function, taking input in a prompt for new batter and initializing it
 - Runs scoring is handled:
 - Runs are added to total score.
 - Runs are credited to the striker.
 - Runs are also added to bowler's conceded runs.
 - Checks if the free hit run was a boundary and updates the batter's stats accordingly.
 - For odd-number runs, strike is rotated using a helper swapstrike() function.
 - Score is refreshed and saved in localStorage.

5. `function extraBall(type)`

- Handles extras like wides and no-balls.
- Increment score and bowler runs.
- NOTE: ball is not counted.
- Updates extras count for wide or no ball.
- For no-balls, prompts user for runs scored on the free hit or if a wicket fell.
- Updates batter and bowler stats accordingly.
- Updates score display and saves game state.

5.3 Byes and leg-byes

When byes or leg-byes function is clicked, it calls `updateExtras(type)` function, where type is bye or legbye, which does the following things;

1. Byes and leg byes are treated as valid deliveries.
2. Then is same as the normal ball. The only difference is that the runs are not counted in the batsman's runs.
3. The string input is converted to an integer using `parseInt()`
4. The runs scored on this all are added in the team's extra runs.
5. `extras.byes` or `extras.legbyes` is incremented appropriately.
6. Ensures strike is rotated on odd runs.
7. `function updateExtras(type)`
 - Handles byes and leg byes extras.
 - Prompts user for runs scored as extras.
 - Adds runs to total score and increments balls and total balls.
 - Note: these runs are not included in batters runs.
 - Other codes is same as of normal legal balls.
 - Updates score display and saves game state.

5.4 Live Commentary

1. `function addcomment(txt)`
 - Adds a timestamped comment to the commentary array by using `new Date().toLocaleTime`
 - Prepends the new comment with the local time of your device with `commentary.unshift(entry);`
 - Updates the commentary in `localStorage`.
 - Calls `rendercomm()` to update the displayed commentary.
2. `function rendercomm()`

- The function retrieves the HTML element `<ul>` with the ID `commentlist`, which will hold the list of commentary entries.
- Before rendering new commentary, the existing commentary is cleared from the container by setting `innerHTML` to an empty string.
- The commentary array is iterated through using `forEach()`
- For each commentary entry (line), a new `<li>` element is created.
- The `textContent` of the `<li>` is set to the commentary entry (line).
- The `<li>` element is appended to the `commentlist` container, which adds the commentary entry to the page.

3. Displaying the Commentary

- When `addcomment(txt)` is called, the commentary is updated, and the `render-comm()` function ensures that all the new commentary entries are displayed in the correct order on the page.
- The recent commentary will always appear at the top of the list due to the use of `unshift()` in `addcomment()`.

4. Main point to keep in mind was you have to add commentary in appropriate function and at appropriate line to get the exact commentary

5. Commentaries are:

(a) Team **X** won the toss and chose to **Y**.

*Where **X** is the team that won the toss, and **Y** is their decision (bat or bowl).*

(b) Team **X** to bat.

*Where **X** is the team batting first, based on the input in `setup.html`.*

(c) **X** played a dot ball.

X scores 1 run.

X scores 4 runs. BOUNDARY!

X scores 6 runs. What a Shot!

X scores **Y** runs.

*Where **X** is the striker, and **Y** is the number of runs scored (selected by the user).*

(d) Extra: Free hit – **X** run(s) scored.

*Where **X** is the number of runs scored during the free hit.*

(e) Extra: Wide by **Z**.

*Where **Z** is the current bowler who bowled the wide.*

(f) Extra: Bye – **X** run(s).

*Where **X** is the number of runs scored as byes.*

(g) Extra: Leg Bye – **X** run(s).

*Where **X** is the number of runs scored as leg byes.*

(h) **Y** is OUT!

*Where **Y** is the striker who got out.*

(i) Z to bowl the next.

Where Z is the next bowler.

(j) Innings over: X scored Y/Z in O overs.

Where X is the batting team, Y is the total runs scored, Z is the number of wickets lost, and O is the total number of overs bowled.

5.5 Chase-Box

1. Chase-box (Need x runs in y balls) also shown in live page.
2. $x = target - score$
3. $y = (overslimit) * 6 - (totalballsplayed)$
4. I did DOM manipulation (Document Object Model manipulation) to emphasis the chasebox -
 - `.innerHTML` is set to the generated chase message.
 - `.style.background`, `.style.color`, and `.style.fontSize` are styled for visibility.
 - `.style.display` is set to "block" to ensure it's shown.

5.6 Background Music

1. background music to setup and summary page is added with `html |audio|` tags.
2. `autoplay` is used to play it automatically as the page is opened
3. `loop` is used to play the music in loop

5.7 More Extensions for Basic Implementations

- Reset button is provided on both scorecard and summary page, and also if a match is not completed yet you open another setup page, you can click on start and it will clear the old data and start a fresh match as i have added `localStorage.clear()` to that button event.
- Extra runs are shown in both scorecard and summary page by saving it in `localStorage`.
- I have used `setTimeout()` function to delay the prompt for smooth working.
- Invalid inputs for extra runs are handled as `if(!NaN())` . If user gives invalid input nothing will change in the data and website will give an alert - ENTER A VALID INPUT. User have to again enter the detail and give a valid input to make it work

6 References

1. W3Schools. “Web Development.”
<https://www.w3schools.com/>
2. International Cricket Council (ICC). “Playing Conditions and Match Regulations.”
<https://www.icc-cricket.com/about/cricket/rules-and-regulations>
3. Cricbuzz. “Live Cricket Score, Stats and Commentary.”
<https://www.cricbuzz.com/>
4. GeeksforGeeks. “JS and HTML(localStorage and JSON.stringify and JSON.parse).”
<https://www.geeksforgeeks.org/>
5. Lecture Slides and Codes. “Basic coding algos.”
<https://mayur.bodhi.cse.iitb.ac.in/courseware/course/4/content/multimedia/coursecontent/>